

# Visualization of LLL and BKZ Algorithms with Vector Representations

Muhammad Imam Chaidar Mujaddid Al-Ghiffari<sup>1[0000-1111-2222-3333]</sup> and  
Amril Syalim<sup>2,3[1111-2222-3333-4444]</sup>

Faculty of Computer Science, Universitas Indonesia

**Abstract.** As the quantum computing starts to threaten the current cryptographic systems such as RSA, cryptographers around the world start to develop the Post-Quantum Cryptography (PQC) algorithms, most of them are lattice-based (for example ML-KEM and ML-DSA), which are strong enough to withstand attacks from quantum-based cryptanalysis techniques such as Shor's algorithm, Grover's algorithm, and other quantum cryptanalysis algorithms. In order to strengthen these PQC algorithms, cryptographers and cryptanalysts alike also developing LLL and BKZ algorithms to test the strength of these PQC algorithm. One of the problems with LLL and BKZ is the fact that while these algorithms are able to penetrate PQC systems, they're harder to learn compared to the algorithms which they succeeded. Sufficient mathematical understanding is required in order to get proper understanding of these algorithms. Some online tools have been available, especially for LLL algorithm, in order to visualize these algorithms so learners can learn them easier. The problem is that many of these visualizations only show the initial and the final result of the input. In this paper, we will build a tool to visualize the processes of both LLL and BKZ algorithms. Through visualization with vectors on Cartesian coordinate as well some panels such as Lovasz status panel and other panels that will explain the current stage of the process of both LLL and BKZ, we expect cryptography learners and enthusiasts alike to understand LLL and BKZ easier.

**Keywords:** Cryptography · Cryptanalysis · Post-Quantum · Lattice · LLL · BKZ.

## 1 Introduction

Cryptanalysis is the technique of analysing and breaking a ciphertext to obtain the plaintext (A. Al-Sabaawi, 2020). The history and the development of cryptanalysis itself can't be separated from the history and development of cryptography. With the onset of the development of quantum computing and subsequently quantum cryptanalysis, many commonly used modern cryptographic algorithms, especially RSA, are threatened by quantum cryptanalysis (). Which is why cryptographers around the world already started to develop cryptographic algorithms that can resist quantum cryptanalysis - the post-quantum cryptography (PQC) (). Most of PQC algorithms that are seen as promising, are lattice-based (Shaik

Ahmadunnisa et al, 2025). For example, Kyber and Dilithium and their standardized versions - the ML-KEM and ML-DSA algorithms. At the same time, cryptanalysts and cryptographers also use these lattice reduction cryptanalysis algorithms in order to evaluate the strength of these lattice-based PQC algorithms to resist their greatest foe - none other than the lattice reduction cryptanalysis algorithms ().

The most common lattice reduction technique is the LLL algorithm, which was already developed long before the advent of quantum computing, and obviously predates the PQC algorithms as well. A polynomial-time algorithm for finding a short and nearly orthogonal basis for a lattice, this algorithm can be used to solve the shortest vector problem (SVP) and the closest vector problem (CVP), which are essential in lattice-based cryptography. We could say that LLL is the foundation of the lattice reduction cryptanalysis algorithms, and to master the subsequent algorithms, mastering the LLL before is a crucial step.

For now, the most promising lattice-based cryptanalysis technique is the BKZ algorithm, which is basically the upscaled version of the LLL algorithm (). Compared to LLL, BKZ can find even shorter vectors in a lattice () and theoretically speaking, BKZ is a far greater threat to lattice-based PQC systems compared to LLL, which are no longer a major threat for these lattice-based PQC systems. To help learners learn LLL and BKZ algorithms, some online solver tools have been developed, for example `fpLLL` and `fpYLLL`, Sage Cell, and `LLL-attack-runner`.

The problems with these LLL and BKZ solvers is that they are either only show the final result, less user-friendly compared to simpler matrix tools, doesn't have visualization, or doesn't show the mathematical processes between the initial state and the final result. The first problem is true for `fpLLL`, `fpYLLL` and `LLL-attack-runner`, as they only show the final result. The second problem is true for Sage Cell, as we can't just insert a "plain" matrix to process it, we have to wrap the matrix within either a Python code, a Sage code, or any code in at least one of languages available in the tool. To put it simply, it isn't as practical as inserting the matrix in some simpler online matrix tools, such as `Calculator.net`, `matrixcalc.org`, or `Desmos`. And the third problem is true for `fpLLL` and `fpYLLL` as well Sage Cell, as they don't even have built-in visualization. Sage Cell can show every step of the LLL process, however, we need to modify the code we inserted in the tool, in a manual way, so we can't use the built-in method of `.LLL()`, and we have to use our own code. For the fourth problem, all previously mentioned LLL and BKZ algorithms solvers have this problem. None of these solvers show the mathematical processes between the initial state and the final result. For Sage Cell, there is no built-in library to show these steps.

So in short, these LLL and BKZ tools aren't practical compared to several online math tools, and they don't automatically provide visualization from the beginning to the final result nor that they provide the step-by-step explanation. Which is actually an important issue, because psychologically wise, humans are generally easier to process information visually (Mayer, 2001) but also how the users interact with them is also an important factor (Christopher D. Hundhausen

et al, 2002). So we definitely need step-by-step explanations just like we need the visualizations, within a system that lets the user to interact with the system from the input insertion to the output visualization. For the visualization part, it's clear that visualization through vectors can significantly help users to understand the process. Why? Because the basis concept for LLL and BKZ is lattice reduction. Which is a linear algebra concept. And it is hard to teach the concepts of linear algebra without vectors (or, to be more precise, vector geometry) visualization (Konyaliouglu et al., 2011).

Our idea is that by visualizing LLL and BKZ algorithms from the beginning of the process until the final result with vectors on 2D and 3D Cartesian coordinates, showing the calculations behind the algorithms, as well some simple theoretical explanations of these algorithms, we can help the users to understand these algorithms easier. The inputs are in the form of matrices. We will develop a tool that not just process LLL and BKZ algorithms, but also show the visualization from the initial state until the final result, with simplified processes shown to help users understand these two algorithms easier. Our main idea for the visualization is through the usage of vectors within a Cartesian field, both in 2D and 3D projections, as well several panels that showing the current process (and its explanation), Lovasz condition status, and a button that can be pressed to go to the previous and next steps. We hope that through these visualization and textual elements, users would have it easier to understand the mechanism of both LLL and BKZ algorithms, which are increasingly becomes more relevant for the security of our data.

## 2 Related Works

### 2.1 Learning with Errors(LWE)

[LWE tuh apa, apa kaitannya sama LLL dan BKZ dan kenapa penting diketahui dan diterapkan di proses LLL dan LWE, cantumkan referensi]

### 2.2 Lattice

A lattice is defined as a set which contains every single linear combinations of vectors from a basis of  $\mathbb{R}^n$ . Therefore, we can state that a lattice is theoretically a discrete subset of vector space with the dimension size of  $n$ .

Let's say, we have  $L$ , which is a lattice, which : -  $k$  is the rank or the dimension size of  $L$  -  $\{v_1, v_2, \dots, v_k\}$  is the basis of  $L$

Then,  $L$  must have vectors of  $v_1, v_2, \dots, v_k$  that each of these vectors are linearly independent and such that  $L = \mathbb{Z}$ -linear span of  $\{v_1, v_2, \dots, v_k\}$ . (Silverman H. Joseph, 2020).

### 2.3 Lattice Reduction

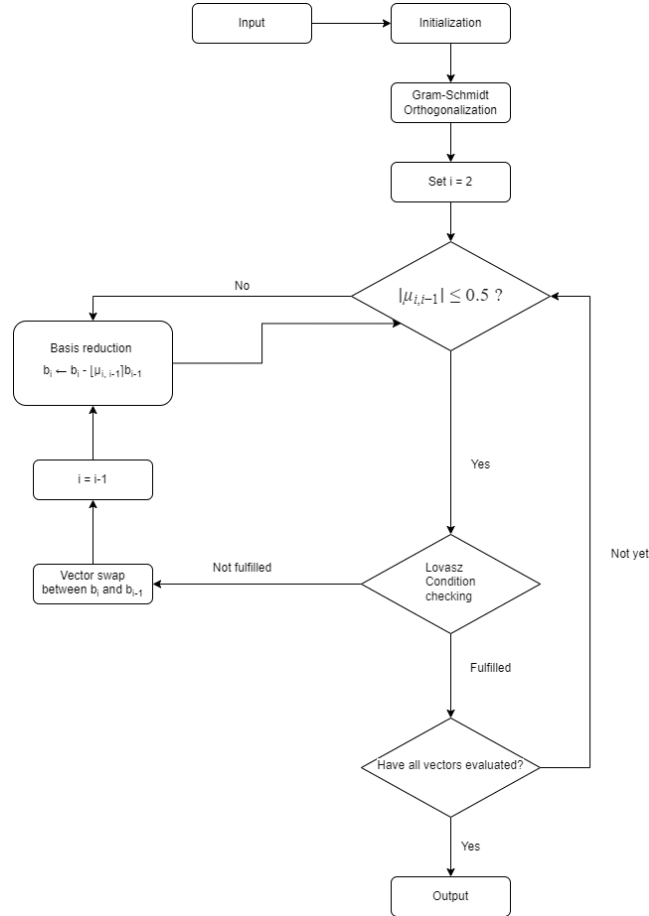
text

Lattice reduction is (Silverman H. Joseph, 2020)

## 2.4 Shortest Vector Problem

The SVP (Shortest Vector Problem) is .... (Silverman H. Joseph, 2020)

## 2.5 LLL



**Fig. 1.** The LLL algorithm

The LLL (Lenstra-Lenstra-Lovasz) cryptanalysis algorithm, developed by Arjen Lenstra, Hendrik Lenstra, and Laszlo Lovasz in 1982, is a [to be completed hehe wkwkwk] [it's for lattice reduction for guessing the encryption keys blablabla]

[hubungkan dengan section sebelumnya]

## 2.6 BKZ

On the other hand, BKZ (Block Korkine-Zolotarev) .... (Ziyu Zhao et al, 2022)

## 2.7 lll-attack-runner

lll-attack-runner is a Javascript-based interactive web application for running advanced lattice basis reduction algorithms such as LLL and BKZ. It is developed by Zachary Robert Bennett with the latest update was in January 2026. Because we aim to create a Javascript-based interactive web application for visualizing lattice-based cryptographic and cryptanalysis algorithms, in which two of them are LLL and BKZ, we will use lll-attack-runner as the backend for our visualization of the LLL and BKZ algorithms. However, since we want to visualize these two algorithms in an intuitive way which is easier to understand and more practical for cryptography learners, we will develop our own frontend for the visualization.

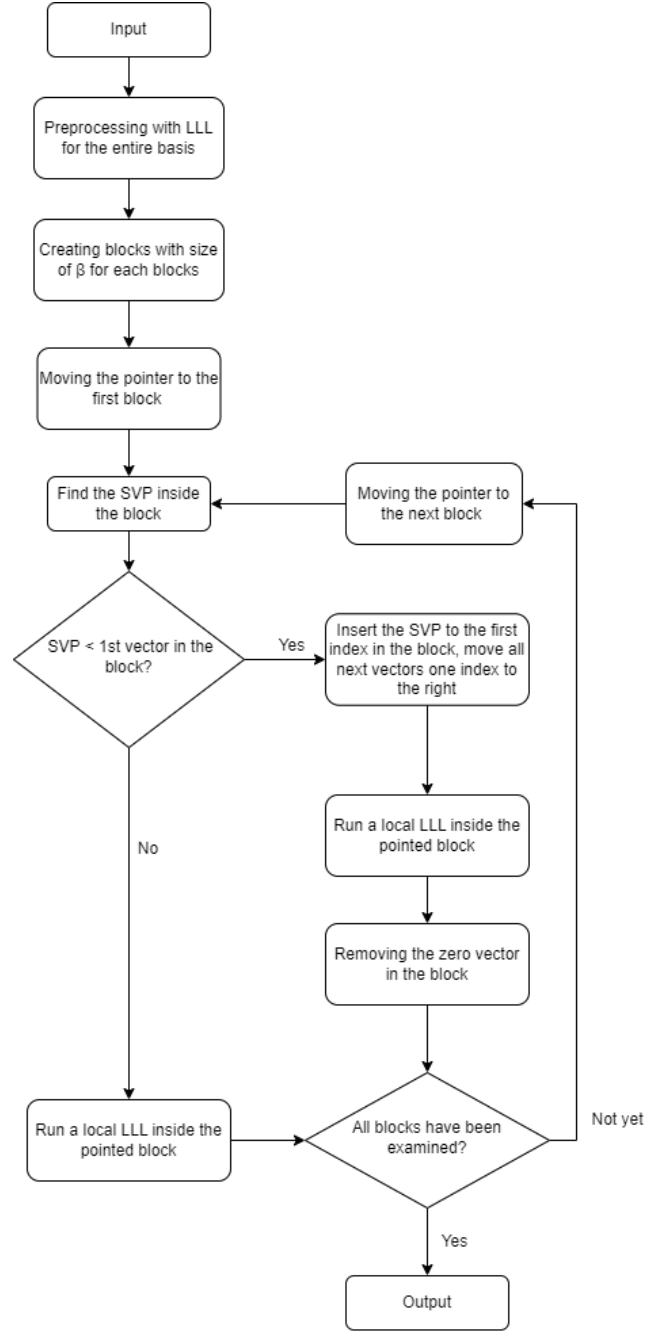
## 2.8 Herramienta de software para la visualización de la criptografía basada en retículas

Authored by Jaziel D. Flores Rodríguez and his fellow colleagues, this research paper was aimed to create a visualization tool for post-quantum cryptographic algorithms. The main focus was to graphically visualize mathematical concepts behind these post-quantum cryptographic algorithms, particularly lattice concepts which are the basis of these algorithms. The tool which was developed in this research, successfully visualizes these lattice-based post-quantum cryptographic algorithms within dimensions of 2, 3, 4, and 5. However, the LLL process for dimensions higher than 3 is still not implemented in this tool, and the BKZ algorithm is also not implemented in this tool. Which is why we will be implementing the LLL process for dimensions higher than 3 in our visualization tool, as well as implementing the BKZ algorithm which is not implemented in this tool.

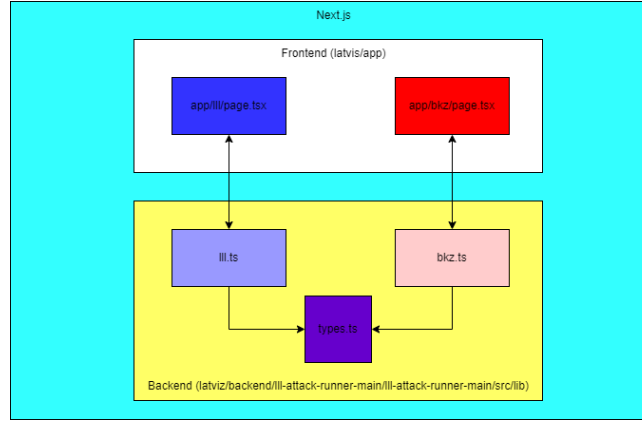
# 3 Architecture

## 3.1 Overview

Within our source code repository, there is a segregation between the backend and frontend. The role of these backend source codes is to process the inputs inserted from the user through the interface at the frontend. And obviously the role of the frontend section, aside from getting the inputs from the user, is to do the entire visualization jobs. Because the visualization is our main priority, we can say that the frontend section is much more complex compared to the backend one.



**Fig. 2.** The BKZ algorithm



**Fig. 3.** Program structure of Latviz's LLL and BKZ visualizers

### 3.2 Backend

We're using Zachary Robert Bennett's `lll-attack-runner`, which is an interactive and Javascript-based lattice-based cryptanalysis solver, as the backend for our visualization of the LLL and BKZ algorithms. We only use three source codes from `lll-attack-runner`, which are : `lll.ts`, `bkz.ts`, and `types.ts`. The `lll.ts`, per its filename, does the LLL algorithm process, and the same goes for `bkz.ts`. Both source codes needs `LLLStep` interface from `types.ts`, and for `bkz.ts`, it also needs `LLLRUN` from `lll.ts`.

### 3.3 Frontend

Within the frontend section, in each of BKZ and LLL algorithm modules, each algorithm has its own folder. Which, in each of these folders, there is a single source code written in Typescript, with naming convention of the algorithm name written in lowercase. Each of these source codes are responsible for running the algorithm process which corresponds to their name. Within each of these source codes, we will use imported functions from Zachary Robert Bennett's `lll-attack-runner` source codes to run the algorithm process.

We will use Next.js, a React-based framework for building web applications, with the goal to create a user-friendly interface as much as we can for our visualization of the LLL and BKZ algorithms. The user will write their own matrix (or copy-and-paste it from any sources), insert needed values (for example, delta value for both LLL and BKZ algorithm and the block size for BKZ algorithm), and the frontend will pass the values inserted by the user into the backend for the core processing. As what we already explained before in the overview section, the frontend's responsibility is mainly for visualization part.

For the visualization part, we will use D3 for the visualization part, combined with React to create matrices and vectors visualizations. Our secondary objective

for the frontend is to create a visualization which have animations for the steps of these algorithms. We believe the animated version of these visualizations can be understand easier by the users.

## 4 Visualization

### 4.1 Vector Representations for Visualizations

Vector visualization is a very essential as well an intuitive concept to gain proper understanding of linear algebra concepts (Calderone Dan, 2023). This also includes LLL and BKZ algorithms. And so, we will use the vector visualization for both 2D and 3D visualization panels in each modules of LLL and BKZ, we will use vectors within 2D and 3D Cartesian fields. Here, every single vector is depicted as an arrow that connects two points from the zero vector (0,0) to a basis vector which already processed from the inserted matrix. Each arrows has different colouring scheme and is unique in terms of value, as every basis vectors are unique. No basis vectors would have more than one arrow that represents them.

### 4.2 Output Visualization and Textual Elements for LLL and BKZ

Visualization of both LLL and BKZ algorithms contains following five visual elements in their output section :

#### 4.2.1 Graph Panel

This is the key visual element. Psychologically speaking, human brain is usually better at understanding visual information compared to textual ones (). We design the graph panel in 2D and 3D projections, both in the fashion of the Cartesian coordinate. Each vector arrows represented in this graph panel have different colour, with their vector written in the same colour as their arrow and placed above the tip of their vector arrow.

#### 4.2.2 Lovasz Status Panel

The panel shows the user of the Lovasz status of the current basis. If the current basis fulfill the Lovasz condition, it will show the user a green check mark and "Satisfied" text, and if not, a red cross mark and "Not Satisfied" text will be shown to the user.

#### 4.2.3 Step and Calculation Details

This panel will tell the user of the current step (for example, "Basis size reduction"). The user can see the details of the calculation in the step by clicking "Show Calculations" button at the upper right corner of this panel.

**4.2.4 Current Basis Panel** Like its namesake, this panel's sole function is to show the user the current basis.





Fig. 4. The 2D visualization

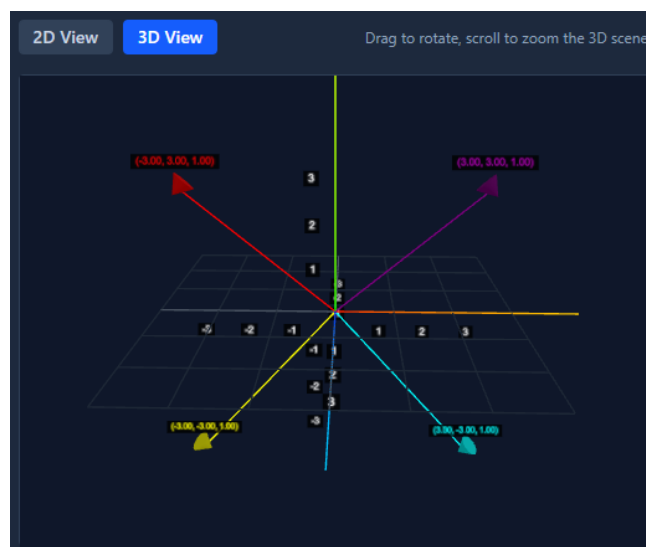


Fig. 5. The 3D visualization



**Fig. 6.** The Lovasz Condition is displayed in this panel, whether it's satisfied (shown here) or not



**Fig. 7.** Current step. Click on the "Show Calculations" for the details

#### 4.2.5 Shortest Vector Panel (BKZ only)

Only for BKZ algorithm, this panel's sole function is to inform the user the shortest vectors found in the current basis. Only shown in the last step.

### 4.3 Visualization of the LLL Algorithm

First, we have the LLL algorithm. The input panel for this algorithm has matrix and delta value insertion fields, and located in the left half of the page. After the user inserted matrix and delta value properly, and clicked "Process Matrix" button, user can see the "Initial Matrix" as well "Initial Basis" panels below the input panel. To the right half of the page, there is the output section, contains four of five visual and textual information panels, without Shortest Vector Panel.

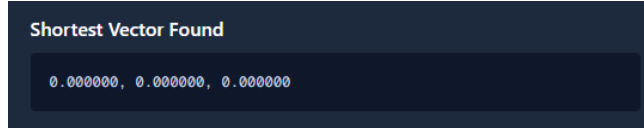
User can click "Back" and "Next" button in the upper part of the output section to see the previous and next steps of the LLL algorithm process. All visual and textual information panels in the output section are subjects of change according to the current step of the LLL algorithm process.

### 4.4 Visualization of the BKZ Algorithm

Next, we have the BKZ algorithm, which is one of the more powerful lattice reduction algorithm. Because theoretically this algorithm could be comprised of

| Current Basis |           |          |
|---------------|-----------|----------|
| 3.000000      | -3.000000 | 1.000000 |
| -3.000000     | 3.000000  | 1.000000 |
| -3.000000     | -3.000000 | 1.000000 |
| 3.000000      | 3.000000  | 1.000000 |

**Fig. 8.** The current basis as with the current step



**Fig. 9.** The SVP we gained, which only shown in the final step of the BKZ process

multiple LLL processes, we design the output section to show the user the number of LLL processes as well an additional information called "Current Process", which provides the user of the steps that is unique to BKZ algorithm, which are the current number of the LLL process and SVP enumeration.

The only differences with LLL page is in the input panel, which has an additional, mandatory field for the block size, and in the output section, in which has Shortest Vector panel. Moreover, in the upper part of the output section, we place four additional information, which are "Block Size", "Delta", "Total LLL Processes", and "Current Process". Out of all of these additional information, only "Current Process" isn't static.

Aside from additional panels and input field, the rest of the visualization and basically mechanism of the output's UI is the same as the LLL ones.

## 5 Result

Please note that the first paragraph of a section or subsection is not indented. The first paragraph that follows a table, figure, equation etc. does not need an indent, either.

Subsequent paragraphs, however, are indented.

### 5.1 LLL Visualization

insert ss of the web here with similar matrix to bkz

### 5.2 BKZ Visualization

insert ss of the web here with similar matrix to lll

## 6 Evaluation

## 7 Conclusion

## References

1. Ziyu Zhao, Jintai Ding. (2022). Practical Improvements on BKZ Algorithm. National Institute of Standards and Technology <https://csrc.nist.gov/csrc/media/Events/2022/fourth-pqc-standardization-conference/documents/papers/practical-improvements-on-bkz-algorithm-pqc2022.pdf>

2. Mayer, RE. (2001). *Multimedia Learning*. Cambridge University Press.
3. Christopher D. Hundhausen, Sarah A. Douglas, John Stasko. (2002). A Meta-Study of Algorithm Visualization Effectiveness. *Journal of Visual Languages and Computing*, 13(3), 259-290.
4. A.Cihan Konyalioglu, Ahmet Isik, Abdullah Kaplan, Seyfullah Hizarci, Merve Durkaya. (2011). Visualization approach in teaching process of linear algebra. *Procedia Social and Behavioral Sciences* 15 (2011) 4040-4044.
5. Silverman, Joseph H. (2020). *An Introduction to Lattices, Lattice Reduction, and Lattice-Based Cryptography*. IAS/Park City Mathematics Series Graduate Summer School Lecture Notes.
6. Calderone, Dan. (2023). *Vector Visualizations*. Github.
7. Author, F., Author, S., Author, T.: *Book title*. 2nd edn. Publisher, Location (1999)
8. Author, A.-B.: *Contribution title*. In: *9th International Proceedings on Proceedings*, pp. 1-2. Publisher, Location (2010)
9. LNCS Homepage, <http://www.springer.com/lncs>, last accessed 2023/10/25

$$x + y = z \tag{1}$$

Please try to avoid rasterized images for line-art diagrams and schemas. Whenever possible, use vector graphics instead